SweetyStoryLab v1.0

Architecture and Development Guide

SweetyStoryLab Engineering

v1.0.0 — First Release

1. Executive Summary

What SweetyStoryLab Is

SweetyStoryLab is a reusable AI storytelling platform, not a single automated Facebook page. It is a **Content Factory**: one shared codebase capable of generating, assembling, and publishing short-form video content across multiple storytelling “brands” (also called “Universes”) — Horror Lab, and eventually Mystery Archive, Bible Stories, History Files, Animal Stories, and others.

The defining architectural rule of the platform is this: **the engine must never know what genre it is generating.** Every brand supplies its own configuration, prompts, branding, voice mapping, fallback content, and Facebook credentials. The engine (`core/`) only ever knows about *pipeline stages* — generate a story, render segments, generate narration, assemble a video, publish, log — never about horror, mystery, or any other genre’s content.

Goals of the Project

- **Modularity** — each responsibility (story generation, image

rendering, voice synthesis, video assembly, publishing) is isolated behind its own interface.
- **Maintainability** — a bug fix or improvement to core mechanics benefits every brand simultaneously, without brand-specific special-casing.
- **Scalability** — the architecture is explicitly designed against the question "will this still work with 50 brands?", not just the two currently in scope.
- **Provider independence** — every external service (LLM, image generation, text-to-speech, publishing) sits behind an abstract interface, so a specific vendor (Groq, Pollinations, ElevenLabs, Facebook) can be swapped without touching brand code or other providers.
- **Configuration over hardcoding** — brand identity (prompts, voice, branding, fallback content) lives entirely in data (YAML, text files, small brand-owned Python modules implementing core-defined interfaces), never hardcoded into the engine.

What v1.0 Accomplished

v1.0 is the successful extraction and validation of Horror Lab — the platform's original and only production brand — out of a single 1,000+ line monolithic script (`post.py`) into the modular `core/` + `brands/horror_lab/` architecture described in this document. It does **not** yet include a second brand; Mystery Archive was deliberately deferred so that Horror Lab's extraction could be fully validated end-to-end first; adding Mystery Archive is the first major goal of the v1.1 cycle.

v1.0 was tagged only after:

- A full line-by-line reconciliation against the real, uploaded `post.py`.
- A formal architecture sign-off review, structured around finding flaws rather than defending prior decisions.
- A release-readiness audit that found and fixed a release-blocking bug (a broken dynamic import path) that had gone undetected through every prior static review.
- A live, end-to-end run in a real local Windows environment, using real credentials, that successfully generated a story, rendered images, synthesized narration, assembled a video, and **published a real Facebook Reel** to the live Horror Lab Page.

---

2. Project Timeline

This section describes the project chronologically. Each phase exists because the previous one exposed a real, concrete need — nothing here was spec'd in advance and built top-down; the architecture emerged from an existing working system and repeated, disciplined validation.

Phase 0 — The starting point. A single working script, `post.py`, already live in production, automating Horror Lab's Facebook Reels: it called Groq for story generation, Pollinations for images, Edge TTS/ElevenLabs for narration, FFmpeg for video assembly, and the Facebook Graph API for publishing, logging results to a local SQLite database, all orchestrated by a GitHub Actions cron workflow. This script worked, but genre, brand identity, and mechanical/engine logic were all fused into one file.

Phase 1 — Vision and platform framing. Before any code was touched, the project was reframed: not "automate Horror Lab" but "build a platform that can run any number of storytelling brands from

one engine.” The explicit test adopted for every subsequent decision was: *“Will this still work if there are 50 brands?”*

Phase 2 — Architectural design (no code). The core/brand boundary was defined on paper first: core owns *behavior* (Story Engine, Renderer Registry, Video Assembly, Providers, Publisher, Storage), brands own *identity* (prompts, branding, hashtags, voice, fallback content, secret references). A `Story`/`Segment` data model was designed to be generic across arbitrary, brand-defined ordered sequences of segments, dispatched through a closed renderer registry rather than an ever-growing `if/elif` chain.

Phase 3 — First extraction pass. `post.py` was split into `core/` and `brands/horror_lab/`, preserving (as intended) Horror Lab’s exact behavior. This first pass was built from an initial reading of the GitHub repository and, in retrospect, contained real inaccuracies — including entirely fabricated functionality (a MusicGen/HuggingFace background-music subsystem that never existed in the real script) and several incorrect constants (wrong LLM model, wrong Facebook API version, wrong subtitle styling values).

Phase 4 — Real-file reconciliation. The actual `post.py` was uploaded and diffed line-by-line against the extraction. Every fabrication and inaccuracy from Phase 3 was found and corrected: the real Groq model and parameters, the real prompts (global/first-person, not the initially-assumed Filipino-specific/third-person version), the real simple local-file music check, the real Facebook API domains/versions/headers, the real SQLite schema, the real subtitle constants. This phase produced `ASSUMPTION_AUDIT.md`, a permanent record of every discrepancy found and how it was resolved.

Phase 5 — Two deliberate architectural decisions. Two design questions were escalated for explicit human decision rather than resolved unilaterally: 1. Narrator voice selection was generalized from a hardcoded `narrator_gender` concept into a fully generic `voice_profile` string on the `Story` schema — an opaque key the brand’s config maps to real voice settings, so a future brand could use `"detective"`/`"witness"` instead of `"male"`/`"female"` with zero core changes. 2. Fallback resilience (what happens when live generation fails) was split so that core owns the *mechanism* (try → parse → validate → fall back) via a new `FallbackProvider` interface, while the brand owns the *content* (the actual fallback story).

Phase 6 — Architecture sign-off review. A formal, critical self-review was performed against six questions: leaking brand knowledge into core, correct layer ownership, premature abstraction, whether a second brand would require core changes, dead/duplicated configuration, and remaining parity risk. This review found and fixed several real issues, including a recurring pattern of core silently defaulting to Horror Lab’s exact values (model, temperature, font, image fallback color) rather than requiring brands to declare them explicitly, and a previously-unverified caption-formatting bug.

Phase 7 — v1.0 release audit. A stricter, blocker-only readiness check (no hypothetical improvements permitted) found the project’s single most serious defect: a dynamic-import path bug that made the pipeline **unable to run at all** via its real entrypoint, despite passing every previous static check. This was traced, evidenced, and fixed.

Phase 8 — Live validation. With real credentials, the pipeline was run in a real local Windows environment. This surfaced three further real bugs invisible to any static review: an FFmpeg subtitle-path escaping failure specific to Windows path syntax, a working-directory side effect from that fix that broke a relative music-file path, and an asset-path consistency gap between how fonts and music files were resolved. Each was fixed one at a time, with evidence and approval required before every change, following the exact “one blocker at a time” discipline established for this phase.

Phase 9 — v1.0 tagged. The pipeline completed a full run — story, images, narration, video, and a real, live Facebook Reel publish — and the release was committed and pushed as `Release v1.0.0: modular Story Engine with first successful Facebook Reel`.

---

3. Architecture Overview

The Core Principle

`core/` is genre-agnostic. It has no knowledge of Horror Lab, horror as a genre, or any specific brand's content. It only knows about generic concepts: a `Story` made of ordered `Segments`, a small closed registry of segment *types* it knows how to render, and a fixed sequence of pipeline stages. `brands/<brand_id>/` supplies everything that makes a brand's output *look and read* like that brand — prompts, parsing logic for that brand's LLM output format, fallback content, branding assets, and secret references.

Module Responsibilities

`core/config/` — Two distinct responsibilities, deliberately kept separate: - `loader.py` resolves **brand-owned** configuration: reads a brand's `config.yaml`, validates required fields (failing loudly if missing — no silent defaults to another brand's values), and resolves Facebook secret *references* (env var names declared in config) against the actual environment. - `platform.py` resolves **platform-owned** infrastructure: currently `GROQ_API_KEY` and `ELEVENLABS_API_KEY`, which are shared across every brand today. This is a deliberate architectural distinction: Facebook credentials vary per brand (each brand has its own Page), so they are brand config; Groq/ElevenLabs do not vary per brand today, so they are platform config. If a future brand needs its own Groq/ElevenLabs account, that one credential moves to the brand-config pattern — the platform module is not expanded preemptively.

`core/story/` — The Story Engine. - `schema.py` defines the generic `Segment` and `Story` dataclasses — the data contract every other module in core operates on. - `parser_base.py` defines the `StoryParser` interface. Core never parses an LLM's raw text output itself — that's entirely brand-owned, because output formats (line-based, JSON, or anything else) are a brand implementation detail, not a core concern. - `fallback_base.py` defines the `FallbackProvider` interface — core's resilience mechanism, brand's content. - `engine.py` orchestrates: calls the LLM provider, hands raw text to the brand's parser, generically validates the resulting `Story` (using the renderer registry's declared required fields per segment type — not a hardcoded scene count), and falls back to the brand's `FallbackProvider` if generation or validation fails.

`core/renderers/` — A closed registry of segment *types* the engine knows how to visually render. - `registry.py` maps a segment type name (e.g. `"narration_scene"`, `"question_slide"`) to a render function and declares each type's required content fields and default timing behavior. - `narration_scene.py` / `question_slide.py` are the actual Pillow-based frame renderers, parameterized entirely by a `RenderContext` (work directory, title, watermark text, font path, scene count) supplied by the brand's config — no hardcoded genre content.

New segment types are added deliberately, at the core level, only when a real brand demonstrates a genuine new rendering need — never invented preemptively for a hypothetical future brand.

`core/providers/` — Provider-independence abstractions. Each subpackage defines a small interface plus one real implementation: - `llm/` — `LLMProvider` interface, `GroqProvider` implementation. - `image/` — `ImageProvider` interface, `PollinationsProvider` implementation (retry logic, local fallback image, brand-supplied style suffix). - `tts/` — `TTSProvider` interface, `EdgeTTSProvider` and `ElevenLabsProvider` implementations, plus `orchestrator.py` (fallback chain logic and subtitle generation — pure mechanics, no brand content). - `music/` — `MusicProvider` interface, `LocalFileMusicProvider` implementation (checks a brand-supplied file path, returns it resolved to an absolute path, or `None`). - `publish/` — `PublishProvider` interface, `FacebookReelsProvider` implementation (the real 3-step Graph API Reels upload flow, with dry-run support for brands without live credentials yet).

`core/video/assembler.py` — Pure FFmpeg mechanics: slideshow assembly, crossfades, subtitle burning, audio mixing. Contains no brand-specific values; all styling (subtitle appearance, timing) is either a true platform-technical constant (e.g. 1080×1920 Reels dimensions) or supplied by the caller.

`core/storage/log_store.py` — SQLite logging, tagging every row with `brand_id` so a shared log store works across brands.

`core/pipeline/` — The composition root. - `run.py`'s `run_brand()` is the one function in the entire system that knows the *sequence* of pipeline stages, and is the **only** place that resolves brand-relative values (asset paths, credentials, provider construction) into concrete objects before handing them to core mechanics. Downstream modules (renderers, assembler, providers) never resolve paths or read config themselves — they only ever receive already-resolved values. - `cli.py` is the real, official entrypoint invoked by GitHub Actions (`python -m core.pipeline.cli --brand <id>`) — deliberately thin, translating `run_brand()`'s outcome into a process exit code (0 = success/dry-run, 1 = crash before completion, 2 = video generated but publish failed) so CI can distinguish failure modes.

`brands/horror_lab/` — Everything genre-specific: `config.yaml` (all brand-owned values), `prompts/` (the real system and user prompts, verbatim from production), `parser.py` (`HorrorLabParser`, the brand's exact line-format parsing logic), `fallback.py` (`HorrorLabFallbackProvider`, the real static fallback story), and `assets/` (font and music files, fetched/placed per-brand).

---

4. Design Philosophy

Separation of concerns. Every module has exactly one reason to change. The Facebook upload mechanics change only if Facebook's API changes; a brand's prompt changes only if that brand's creative direction changes; these are independent axes of change, and the architecture keeps them independent in code as well as in intent.

Dependency direction. Brands depend on core (importing its interfaces and schema); core never depends on brands. This is enforced structurally, not just by convention: `core/` contains no import of anything under `brands/`, anywhere. The one place brand code is ever reached from core is a single, deliberate seam — dynamic import via `importlib.import_module()`, driven entirely by a dotted-path *string* read from the brand's own config. Core doesn't know which brand it's running until that string is read at runtime.

Brand isolation. A brand folder is a self-contained unit: its config, prompts, parsing logic, fallback content, and assets. Adding a brand should — and, per the Mystery Archive extension test performed

during the architecture review, does — require only adding files under `brands/<new_brand>/`, never modifying `core/`.

Provider abstraction. Every external, potentially-swappable service sits behind a narrow interface (`LLMProvider`, `ImageProvider`, `TTSProvider`, `MusicProvider`, `PublishProvider`). This was not a speculative decision — it was justified from the start of the project by an explicit, stated requirement to use free-tier services that are individually unstable and likely to be swapped over the platform's lifetime.

Configuration-driven design. Any value that could plausibly differ between brands is configuration, not code — including things that only have one value today (e.g. Horror Lab's image style suffix, or its dark fallback frame color), because leaving them hardcoded in core was repeatedly found, during review, to silently leak one brand's identity into what should be genre-blind code.

The composition root pattern. `run_brand()` is the single place where abstract pieces become concrete: where a brand-relative path becomes an absolute path, where a dotted config string becomes an instantiated Python object, where a `voice_profile` string becomes an actual TTS voice name. Every other module downstream of it — renderers, providers, the assembler — receives only fully-resolved values and performs no resolution of its own. This was arrived at directly through debugging experience (see Section 10) rather than decided in advance: font paths and music paths were originally resolved inconsistently (one hardcoded partially in `run.py`, the other passed through with an unresolved relative reference), which caused a real, live production bug. The fix was to unify both under the same one-owner rule.

Reasoning Behind Key Ownership Decisions

- **Facebook secrets are brand config; Groq/ElevenLabs secrets are platform config** — because the former genuinely varies per brand (each brand has its own Page) and the latter does not, today. This is a deliberately reversible decision, documented directly in `core/config/platform.py`'s own docstring, with the explicit trigger condition for reversing it stated in the code itself.
- **The story parser is brand-owned, not a generic core capability** — a JSON-based, fully generic parsing protocol was proposed and initially built during Phase 3, then deliberately reverted: mixing an extraction/migration effort with a protocol redesign made both harder to validate independently. Keeping Horror Lab's real, currently-hardcoded parsing format isolated behind a `StoryParser` interface preserves both the platform's genre-blindness and Horror Lab's exact production behavior, and defers "should parsing be generic" to its own, separately-decidable milestone.
- **Fallback resilience mechanism is core; fallback content is brand** — because "try, validate, fall back" is pipeline resilience that every brand will eventually want, while the actual fallback story is unmistakably brand content.
- **`platform.py` stays inside `core/config/`, not a top-level `platform/` package** — considered during the architecture review and explicitly rejected as premature. The concrete trigger for splitting it out was defined precisely: the first time something needs state or coordination *across* brand runs (a shared-quota rate limiter, a fleet-wide scheduler, a cross-brand policy gate) — not merely "more secrets accumulate here." No such need exists yet.

---

5. Folder Structure

```
sweetystorylab/
├── core/                              # Genre-agnostic engine
│   ├── config/
│   │   ├── loader.py                  # Brand config.yaml loading +
validation
│   │   └── platform.py                # Platform-wide shared
secrets (Groq, ElevenLabs)
│   ├── story/
│   │   ├── schema.py                  # Generic Segment / Story
data model
│   │   ├── parser_base.py             # StoryParser interface
(brand implements)
│   │   ├── fallback_base.py           # FallbackProvider interface
(brand implements)
│   │   └── engine.py                  # generate_story():
orchestration + fallback mechanism
│   ├── renderers/
│   │   ├── registry.py                # Closed segment-type
registry
│   │   ├── narration_scene.py         # Renders a narration scene
frame
│   │   └── question_slide.py          # Renders a closing question
frame
│   ├── providers/
│   │   ├── llm/                       # LLMProvider + GroqProvider
│   │   ├── image/                     # ImageProvider +
PollinationsProvider
│   │   ├── tts/                       # TTSProvider +
EdgeTTS/ElevenLabs + orchestrator
│   │   ├── music/                     # MusicProvider +
LocalFileMusicProvider
│   │   └── publish/                   # PublishProvider +
FacebookReelsProvider
│   ├── video/
│   │   └── assembler.py               # FFmpeg
slideshow/crossfade/subtitle/audio mix
│   ├── storage/
│   │   └── log_store.py               # SQLite result logging
│   └── pipeline/
│       ├── run.py                     # run_brand() — the
composition root
│       └── cli.py                     # Official entrypoint, thin,
maps result -> exit code
│
├── brands/
│   └── horror_lab/                    # Everything genre-specific
to Horror Lab
│       ├── config.yaml                # All brand-owned
configuration values
│       ├── prompts/
│       │   ├── system_prompt.txt      # Real production system
prompt, verbatim
│       │   └── user_prompt.txt        # Real production user
prompt, verbatim
│       ├── parser.py                  # HorrorLabParser
(StoryParser implementation)
│       ├── fallback.py                # HorrorLabFallbackProvider
(FallbackProvider implementation)
│       └── assets/
│           ├── fonts/
│           │   └── Lora-Italic.ttf    # Fetched by workflow, or
placed manually for local runs
│           └── sounds/
│               └── suspense.mp3       # Local background music
```

```
fallback
│
├── .github-workflows-horror_lab-post.yml   # -> repo's
.github/workflows/post.yml
├── ASSUMPTION_AUDIT.md                     # Phase 3/4 extraction
reconciliation record
└── RELEASE_AUDIT.md                        # Phase 7 release-
readiness findings & resolutions
```

Why this shape, specifically: `core` and `brands` are siblings, both top-level Python packages relative to the repository root — this is not incidental; it is exactly the convention the real CLI invocation (`python -m core.pipeline.cli`) and every dynamic `importlib.import_module("brands.<id>.<module>")` call depend on. (Section 10 documents a real production-blocking bug that occurred when two brand files used a different, incompatible import convention.)

---

6. End-to-End Pipeline Walkthrough

Starting from:

```
python -m core.pipeline.cli --brand horror_lab --db-path
horror_log.db
```

1. CLI parses arguments (`core/pipeline/cli.py`) — `--brand`, `--brands-root` (default `brands`), `--db-path`. Calls `run_brand()`, wraps it in a single `try/except`, and maps the outcome to a process exit code.

2. Configuration loading (`core/config/loader.py::load_brand_config()`) — reads `brands/horror_lab/config.yaml`. Validates every required field is present (segment template, prompt file paths, parser/fallback module paths, `llm.*`, `branding.font`) — missing any of these raises a `ConfigError` immediately, rather than silently defaulting to another brand's values. Resolves `facebook.page_id_env`/`access_token_env` against the actual environment; if either is unset, `is_dry_run` is set `True`. Also validates that `voice.default_profile`, if set, actually exists as a key in `voice.profiles` — a mismatch here fails at load time, not deep inside a later TTS call.

3. Prompt loading — `run_brand()` reads the brand's `system_prompt.txt` and `user_prompt.txt` as raw, opaque strings.

4. Provider/parser/fallback construction — `GroqProvider` is constructed using the **platform-level** Groq key (`core/config/platform.py`) and the **brand-level** model/temperature/max_tokens from config. The brand's `parser_module` and `fallback_provider_module` dotted-path strings are dynamically imported and instantiated.

5. Story generation (`core/story/engine.py::generate_story()`) — calls the LLM provider, hands the raw text to the brand's `StoryParser.parse()`, and generically validates the resulting `Story` by checking, for every segment, that its type's required content fields (declared in the renderer registry) are actually populated. If the LLM call raises, or the resulting story fails validation, the brand's `FallbackProvider.get_fallback_story()` is used instead — the pipeline never hard-fails here.

6. Per-segment rendering — `run_brand()` iterates the `Story`'s segments in order. For each segment with an `image_prompt`, `PollinationsProvider` is called (brand-supplied style suffix appended, 3

retries with backoff, brand-supplied local fallback image directory, then a brand-supplied plain fallback color as a last resort). Segments without their own `image_prompt` (e.g. the closing question slide) reuse the most recently generated image. Each segment is then dispatched through `core/renderers/registry.py` to its type's render function, producing a JPEG frame.

7. Narration generation (`core/providers/tts/orchestrator.py::generate_voice()`) — the `Story`'s `voice_profile` (an opaque brand-defined key) is looked up in the brand's `voice.profiles` config to resolve actual voice settings. `ElevenLabsProvider` is tried first if a platform ElevenLabs key exists; on failure or absence, `EdgeTTSProvider` is used as a guaranteed-available fallback. Subtitle timing is generated either from ElevenLabs' word-boundary data or estimated from narration length.

8. Music resolution (`core/providers/music/local_file_provider.py`) — the brand's `music.local_file` (a brand-relative path, resolved to absolute by `run_brand()` before construction) is checked for existence; returns the absolute path or `None` (no music) — never raises.

9. Video assembly (`core/video/assembler.py::build_video()`) — assembles the rendered frames into a slideshow with crossfades, burns in subtitles (using only the subtitle file's basename with the FFmpeg subprocess's working directory set to the segment work folder — see Section 10 for why), mixes in background music if present, and produces the final MP4.

10. Publishing (`core/providers/publish/facebook_provider.py::FacebookReelsProvider.publish()`) — if `is_dry_run` is `True`, returns a dry-run result immediately with no network call. Otherwise performs the real 3-step Graph API v21.0 Reels upload (start → binary upload to `rupload.facebook.com` → finish/publish), including the required delay between upload and publish phases.

11. Logging (`core/storage/log_store.py`) — writes one row (brand ID, timestamp, title, video ID, status, error) regardless of outcome.

12. Exit code — the CLI reports success (0) for both a real publish and a dry run, exit code 2 if the video was generated but publishing failed, and exit code 1 for any crash before video completion.

---

7. Configuration Reference

Every field currently read from `brands/<id>/config.yaml`, who owns it, and where it's consumed.

Field	Owner	Consumed b
`brand.id`, `brand.name`	Brand	`core/config/loade` (identity), used as `brand_id` throughe the pipeline and i logging
`brand.emoji`, `brand.parent`	Brand	Loaded and valid but **not currentl consumed by ar pipeline logic** — descriptive identi metadata only
`content.system_prompt_file`, `content.user_prompt_file`	Brand	`run.py` — read as text, passed to th provider

`content.parser_module`	Brand	`run.py` — dynamic imported to const the brand's `Story`
`content.fallback_provider_module`	Brand	`run.py` — dynamic imported to const the brand's `FallbackProvider`
`content.segment_template`	Brand	`core/story/engine` validation logic references segme types via the rend registry; the actu ordering is enforc the brand's parse output today, not directly from this during a run — it primarily docume the shape brand content must pro (see Section 12 fo planned tightenir
`llm.model`, `llm.temperature`, `llm.max_tokens`	Brand	`run.py` → `GroqProv` construction and `generate_story()` **Required** — no c level default
`voice.profiles`, `voice.default_profile`	Brand	`run.py` — resolves `Story.voice_profi` actual TTS settin Validated at confi time
`branding.watermark_text`	Brand	`run.py` → `RenderCo` → both renderers
`branding.font`	Brand	`run.py` — brand-r path, resolved via `os.path.join(brar config.font_file)`
`branding.font_source_url`	Brand (documentation only)	Not read by any I code — informs whoever writes th brand's GitHub A workflow what to
`music.local_file`	Brand	`run.py` — brand-r path, resolved the same way as `branding.font`, pa to `LocalFileMusicPro`
`image.style_suffix`	Brand	`run.py` → `PollinationsProvi` appended to ever LLM-generated i prompt
`image.fallback_dir`	Brand	`PollinationsProvi` local fallback ima directory if the li fails after retries
`image.fallback_color`	Brand	`run.py`'s exceptio handler — last-re plain frame color image generation completely

`facebook.page_id_env`, `facebook.access_token_env`	Brand	`core/config/loade` — names of environment vari to resolve (not th secrets themselv
`schedule.times_pht`	Brand (documentation only)	Not read by any I code — schedulin owned entirely by GitHub Actions workflow's cron t

Platform-level (not in any brand's `config.yaml`):

Variable	Owner	Consumed by
`GROQ_API_KEY`	Platform	`core/config/platform.py`
`ELEVENLABS_API_KEY`	Platform (optional)	`core/config/platform.py`

8. Providers

LLMProvider (`core/providers/llm/`) — `generate(system_prompt, user_prompt, temperature, max_tokens) -> str`. One implementation: `GroqProvider`. To add a new LLM backend (e.g. a different vendor, or a local model), implement this interface and change which class `run.py` instantiates — no other file changes.

ImageProvider (`core/providers/image/`) — `generate(prompt, output_path, index) -> str`. One implementation: `PollinationsProvider`, with its own internal retry/fallback chain. A new image backend implements this same interface; the brand-supplied `style_suffix` mechanism is generic and would apply to any implementation.

TTSProvider (`core/providers/tts/`) — `generate(text, voice_file) -> bool`. Two implementations, chained by `orchestrator.py`: `ElevenLabsProvider` (primary, if a platform key exists) and `EdgeTTSProvider` (fallback, always available, no API key required). A new voice backend can be added to the chain without altering the chain's fallback logic itself.

MusicProvider (`core/providers/music/`) — `generate(*args, **kwargs) -> str | None`. One implementation: `LocalFileMusicProvider`, deliberately simple (checks a file exists, returns its absolute path). A future brand needing generated (rather than pre-made) music would implement this same interface without changing how `run.py` calls it.

PublishProvider (`core/providers/publish/`) — `publish(video_path, title, caption) -> PublishResult`. One implementation: `FacebookReelsProvider`, with built-in dry-run support. Adding a second platform (YouTube Shorts, TikTok) means implementing this interface and having a brand's config select which publisher(s) to use — this is a natural v1.1+ extension point, not yet built.

Adding a new provider, generally: implement the relevant `*Provider` abstract base class in a new file under the appropriate `core/providers/<kind>/` subpackage; update the one construction call site in `run.py` (or, for a future multi-provider-per-brand design, make that selection config-driven). No renderer, no other provider, and no brand code needs to change.

9. Brand Development Guide

To add a completely new brand (e.g. Mystery Archive), based directly on the extension test performed during the architecture review:

1. **Create `brands/<new_brand_id>/`** with the same shape as `brands/horror_lab/`.
2. **Write `prompts/system_prompt.txt` and `user_prompt.txt`** — the brand's actual creative direction and output-format instructions for the LLM.
3. **Write `parser.py`** — a class implementing `core.story.parser_base.StoryParser`, whose `parse(raw_text) -> Story` understands whatever output format the brand's prompt actually produces. This can reuse Horror Lab's line-based format, or use a different one entirely — core does not care.
4. **Write `fallback.py`** — a class implementing `core.story.fallback_base.FallbackProvider`, returning a complete, hand-written `Story` to use if live generation fails.
5. **Provide assets** — `assets/fonts/<font>.ttf` and, optionally, `assets/sounds/<file>.mp3`.
6. **Write `config.yaml`** — every field documented in Section 7: identity, prompt/parser/fallback module paths, segment template, required `llm.*` values, `voice.profiles/default_profile`, branding, music, image style/fallback, and Facebook secret *references* (new env var names — a new brand means new GitHub secrets, since each brand has its own Page).
7. **Add a GitHub Actions workflow** (`.github/workflows/<new_brand>-post.yml`) — its own cron schedule, its own font-fetch step (if applicable), its own secrets injection, and a CLI invocation of `python -m core.pipeline.cli --brand <new_brand_id> --db-path <new_brand>_log.db`.
8. **Add the new brand's GitHub secrets** — Page ID and Page access token, following the naming convention established for Horror Lab.

No step above requires modifying anything under `core/`. This was explicitly verified, not assumed, as part of the architecture sign-off review.

---

10. Major Debugging Journey

10.1 — Dynamic import path failure (release blocker)

Root cause: `brands/horror_lab/parser.py` and `fallback.py` imported core modules via `from sweetystorylab.core.story...` — assuming a top-level package named `sweetystorylab`. Every other module in the system (including the CLI invocation itself and `run_brand()`'s own dynamic `importlib.import_module("brands.horror_lab.parser")` call) assumes `core` and `brands` are top-level packages relative to the working directory. These two conventions are mutually incompatible under the real invocation.

Investigation: every prior static check (syntax compilation, direct `import core.pipeline.run`) passed, because none of them exercised the dynamic import path — that only fires when a brand's parser is actually instantiated at runtime. The bug was found only by literally running the real CLI command and reading the resulting traceback, which pinpointed the exact failing import line.

Final solution: four import lines changed, across exactly two files, to use `core.story...` instead of `sweetystorylab.core.story...`.

Why architecturally correct: this did not touch the core/brand interface boundary at all — it was purely a matter of how two files located already-correctly-designed interfaces. The fix aligned the outlier files with the convention every other file already used correctly.

10.2 — Windows FFmpeg subtitle path parsing

Root cause: FFmpeg's `subtitles=` filter argument is parsed by splitting on `:` (to separate filter options) and treating `\` as an escape character. A Windows absolute path (e.g. `C:\Users\...\voice.srt`) contains both unescaped, causing FFmpeg to misparse the drive letter and path separators as filter option syntax.

Investigation: first attempted a generic escaping transformation (`\` → `/`, `:` → `\:`), verified to be a true no-op on POSIX-style paths (proving it introduced no OS-specific branching) — but this proved insufficient for the `subtitles` filter's specific positional-argument parsing behavior, which still misassigned the escaped path.

Final solution: eliminated the ambiguity structurally rather than perfecting the escaping — the FFmpeg subprocess's working directory is set to the segment's temp work folder, and only the subtitle file's bare basename (no path, no colon, no backslash) is passed to the filter.

Why architecturally correct: rather than iterating indefinitely on escaping edge cases, this removes the class of problem entirely; it required no OS detection or branching, and is provably safe on any platform.

10.3 — Asset path ownership inconsistency (regression)

Root cause: the working-directory change from 10.2 had a side effect: a *different* relative path — the brand's configured local music file (`sounds/suspense.mp3`) — was resolved correctly at the point it was checked for existence (while the working directory was still the repo root), but that relative string was then used again later, inside the now-relocated FFmpeg subprocess, where it no longer pointed anywhere valid.

Investigation: traced precisely to a working-directory framing mismatch between when a path is *verified* and when it is later *used*.

Final solution, in two parts: first, `LocalFileMusicProvider` was changed to resolve and return an absolute path at the moment it verifies the file exists, rather than passing the original relative string through. Second, this was used as the occasion to also unify how font and music assets are resolved: both are now specified in `config.yaml` as full brand-relative paths (`assets/fonts/...`, `assets/sounds/...`) and resolved identically by `run_brand()` — `os.path.join(brand_dir, config.<field>)` — removing a previously-hardcoded `"assets"` literal that had lived only in the font-resolution code path.

Why architecturally correct: this generalized a fix rather than patching the one symptom, applying the existing composition-root principle (only `run_brand()` resolves brand-relative paths; every downstream module receives already-resolved values) consistently to both asset types instead of leaving an inconsistency between them.

10.4 — Local testing vs. GitHub Actions

Production has always run on Ubuntu via GitHub Actions; this was the first time the pipeline was exercised on a Windows development machine. Two of the three bugs in this section (10.2, 10.3) were Windows-path-specific and would very likely never have surfaced on the actual production OS. The explicit decision made at this point was to validate on GitHub Actions first when possible, since it matches production exactly — but since local Windows testing was already underway and provided faster iteration, both were fixed anyway, since portability (not just Ubuntu-only correctness) was judged a legitimate goal for `core/` regardless of where production currently runs.

10.5 — Facebook authentication

Long-lived Page access tokens were obtained via the standard Meta flow: a short-lived User token from Graph API Explorer, extended via the Access Token Debugger to a long-lived User token (~60 days), then converted to a Page token via a `me/accounts` request. A key operational fact documented here for future reference: **Facebook does not provide any way to retrieve a previously-generated token's value again** — if a token string is lost without being saved, it must be regenerated (this does not invalidate the old one; multiple valid tokens can coexist).

10.6 — Runtime validation gaps closed during the architecture review

Beyond the four issues above, the architecture review found and fixed a class of latent bugs before they could reach production: core silently defaulting to Horror Lab's exact real values (Groq model/temperature/max_tokens, font filename, image fallback color, image fallback directory) when a brand's config omitted them, rather than failing loudly. Also fixed: a caption-formatting mismatch against real production output (missing the double-newline separator between caption text and hashtags), and a voice-profile misconfiguration that previously would have failed silently deep inside a TTS subprocess call rather than at config-load time. All of these were converted to fail-fast validation errors rather than silent wrong behavior.

---

11. Important Design Decisions

- **Rejected:** a fully generic, JSON-based LLM output protocol for all brands. **Chosen instead:** brand-owned parsing behind a core-defined interface. Reason: mixing a migration/extraction effort with a protocol redesign made both harder to validate; the generic protocol remains a legitimate future milestone, deliberately deferred rather than abandoned.
- **Rejected:** `narrator_gender` as a hardcoded concept anywhere in core. **Chosen instead:** a fully opaque `voice_profile` string, interpreted only by brand config. Reason: a future brand's narrator concept (e.g. character roles rather than gender) should require zero core changes.
- **Rejected:** treating fallback story content as part of the parser's responsibility. **Chosen instead:** a separate `FallbackProvider` interface. Reason: parsing and resilience are different concerns with different owners (parsing format is inherent to the brand's prompt design; resilience is a pipeline-wide guarantee every brand should get uniformly).
- **Rejected:** moving `core/config/platform.py` into a new top-level `platform/` package now. Reason: no responsibility exists today that actually requires cross-brand coordination; the move would change file organization with zero behavioral benefit, which was explicitly

identified as premature abstraction during review.
- **Rejected:** OS-specific branching to fix the Windows FFmpeg subtitle bug. **Chosen instead:** a structural fix (working directory + basename) that is correct on every platform by construction, not by conditional.
- **Rejected:** leaving brand-specific values as core-level defaults "since only one brand exists today." **Chosen instead:** required fields that fail loudly when a brand's config is incomplete. Reason: found, during review, to be a repeating pattern that had already caused two separate near-misses (a fabricated-value bug and a real production formatting bug), and was judged to represent a systemic gap in how "core must never assume brand content" had been enforced.

---

12. Future Improvements (v1.1 and Beyond — NOT Implemented)

The following are explicitly **not** part of v1.0 and should not be assumed to exist:

- **Mystery Archive as the second brand** — the actual validation of "add a brand via config only, zero core changes," which v1.0 enabled but did not itself perform.
- **A generic, JSON-based `StoryParser` implementation**, offered as an option brands can adopt instead of writing their own line-format parser, without removing the brand-owned-parser capability itself.
- **A real `segment_template` enforcement mechanism** — today the field primarily documents intended shape; the parser's actual output is what determines the real segment sequence. Enforcing that the parser's output matches the declared template would be a genuine correctness improvement.
- **Multi-platform publishing** (YouTube Shorts, TikTok) via additional `PublishProvider` implementations, with brand config selecting which platform(s) to target.
- **Cross-brand rate-limit coordination** for the shared platform-level Groq/ElevenLabs accounts, once multiple brands' workflows can run concurrently — the concrete trigger condition already identified for eventually promoting `platform.py` into its own package.
- **A per-brand or fleet-wide dashboard**, which would be the first real consumer of the currently-unused `brand.emoji`/`brand.parent` identity metadata fields.
- **Reconsidering per-brand GitHub Actions workflow duplication** — acceptable at the current brand count, flagged early as a likely concern once the platform reaches many brands (a reusable `workflow_call` pattern was suggested as a future direction, not built).

---

13. Lessons Learned

Static verification is necessary but not sufficient. Every check performed before live execution — diffing against the real source file, syntax compilation, even direct module imports — passed cleanly right up until the actual CLI command was run for the first time, which immediately failed on an import-path bug none of those checks could have caught, because none of them exercised the dynamic-import code path that only fires at real runtime. The lesson generalized from this: any code path reached only through a string (a dynamically

constructed import, a config-driven dispatch) is invisible to ordinary static analysis and needs to be exercised directly, not just reasoned about.

Claiming a fix was applied is not the same as verifying it was. At one point during live validation, a proposed one-line fix was described as “already done” in conversation when it had, in fact, only ever been proposed — not written to the actual file. The failure this caused looked identical to a genuinely new bug until the actual file content was checked directly. The corrective practice adopted afterward — verifying a change’s real, current effect (e.g. by actually running the fixed function and inspecting its output) rather than asserting it — proved necessary, not optional.

A pattern, once found, tends to recur elsewhere until it is treated as a principle, not a one-off patch. The “core silently defaults to one brand’s real value” defect was found and fixed individually at least four separate times (an accent color, a music file path, an LLM model default, a font filename default) before being recognized as a single systemic gap rather than four unrelated bugs. Fixing instances as they’re found is necessary, but insufficient — the underlying rule they all violate needs to be named and checked for exhaustively once identified, rather than assumed fixed after the first correction.

Environment differences are a real, distinct category of risk from logic errors. Two of the four debugging-journey issues in this document (10.2, 10.3) were specific to running on Windows rather than the actual Ubuntu production environment, and would very plausibly never have been discovered by code review alone, no matter how careful. Live validation, in an environment as close to production as practical, surfaced real defects that a purely static, however rigorous, review did not and structurally could not.

Separating “does this reveal a bug” from “should we redesign” kept the debugging process tractable. Each live-validation failure was deliberately classified — environment issue, runtime bug, production parity issue, or release blocker — before any fix was proposed, and fixes were scoped to the smallest change that resolved the specific evidenced failure, explicitly deferring any tempting broader refactor. This discipline (one blocker at a time, evidence before diagnosis, approval before code changes) is what allowed four real, non-obvious bugs to be found and fixed in sequence without the process collapsing into speculative, unverified rewriting.